LOCUS: Integrated Storage-Separated Private-Key Recovery with Selectable Threshold Suites

Anonymous Author(s)

Abstract

Long-lived private keys create a recovery dilemma: a mechanism must remain usable after device loss, yet a leaked backup must not become a local oracle for testing predictable human input. LOCUS is an integrated recovery system that composes deterministic, versioned processing of structured recovery input with an explicitly selected threshold recovery suite and storage-separated authenticated encryption. The active client alone converts structured input through a CuePolicy, derives suite-bound password material, obtains a high-entropy recovery secret through either the Yi threshold password-authenticated secret-sharing suite or an augmented password-protected secret-sharing (aPPSS) suite, and derives an AES-256-GCM wrapping key. The cloud role stores only the encrypted private-key backup and public authenticated configuration; five separately identified authorizers hold suite-specific state, with paired 2-of-3 and 3-of-5 reconstruction profiles and a distinct 4-of-5 authorization quorum.

The reference implementation connects a loopback Manager, a narrowly scoped container controller, dynamically created clean Client interfaces, authenticated admission and discovery, a storage gateway and local S3-compatible provider, a resolver, and five party services. A signed descriptor and live party summaries bind one suite, policy, membership, and epoch without recovery-time fallback. Aggregate retained experiments exercise 42 persistent-state scenarios and 30 payload-free application-flow scenarios over the managed graph. A separately versioned performance methodology specifies 324 scheduled observations across four matched suite/topology arms; its numerical results remain provisional in this personal draft. The resulting claim is deliberately narrow: cloud-only, below-threshold party, and matching cloud-plus-below-threshold persistent states do not expose a local offline cue-testing predicate under the inherited suite assumptions. LOCUS does not claim new threshold cryptography, cue memorability, global rollback-resistant attempt control, independent administration, or production readiness.

CCS Concepts

• **Security and privacy → Key management**; **Security protocols**; *Distributed systems security*.

Keywords

private-key recovery, threshold password-authenticated secret sharing, password-protected secret sharing, cloud backup, clean-client recovery, storage separation

ACM Reference Format:

Anonymous Author(s). 2027. LOCUS: Integrated Storage-Separated Private-Key Recovery with Selectable Threshold Suites. In *Proceedings of the 22nd ACM Asia Conference on Computer and Communications Security (ASIACCS '27), July 12–16, 2027, Macau, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

Personal provisional status draft. This document is not the authoritative LOCUS manuscript, does not modify the project plan or claim matrix, and is not a source of evidence. It presents the intended completed-system narrative so that the project can be reviewed as a whole. Retained P8 evidence is reported only at its established scope. Unavailable P9 measurements, independent-review outcomes, and final artifact digests are marked **TBD** rather than invented.

1 Introduction

Private keys protect encrypted archives, end-to-end encrypted accounts, identities, credentials, and self-custodied assets. Their security depends on exclusive control, but that same exclusivity makes loss catastrophic. Recovery systems generally relax one of three properties. Custodial systems concentrate recovery power in one operator. High-entropy recovery codes preserve cryptographic strength but create another long-lived object that must be kept available. Password-encrypted backups remain convenient but can become offline-guessing targets when ciphertext and a password verifier are copied. Social and threshold recovery distribute authority, but often leave the human-input, discovery, encrypted-storage, and replacement-device workflows outside the evaluated protocol.

LOCUS studies a different systems composition. A client deterministically converts structured recovery input into exact bytes. Those bytes are not used directly as an encryption key and are not stored as a verifier. Instead, they mediate an online threshold protocol that yields high-entropy recovery material. The client derives an authenticated-encryption key from that material, while an object-storage role keeps only the encrypted private-key backup and public authenticated configuration. The intended persistent-state boundary is therefore:

$$\text{cloud} \perp \text{threshold state} \perp \text{complete client secret path}.$$

The design is useful only if the full replacement-device path preserves this boundary. It must authenticate discovery, descriptors, party identities, admission, storage operations, and lifecycle transitions without reintroducing a cue-derived predicate.

Structured input is not automatically memorable or unpredictable. Studies of personal-knowledge questions, graphical cues, and location-based fallback authentication show both the attraction and the hazards of such interfaces [2, 4, 5, 16]. LOCUS therefore separates two questions. A CuePolicy is an engineering contract for deterministic bytes, ambiguity, duplicates, resolver behavior, and

ASIACCS '27, Macau, China

ACM ISBN 979-8-XXXX-XXXX-X/27/07
https://doi.org/10.1145/XXXXXXX.XXXXXXX

versioning. Whether people can reliably remember or safely select inputs for a particular policy is an empirical usability question that this work does not answer.

The current system supports two independently selected inherited recovery suites. The frozen Yi TPASS suite follows Yi et al. [26, 27]. The second suite instantiates the Figure-4 augmented password-protected secret-sharing construction of Dziembowski et al.'s *Password-Protected Threshold Signatures*, using an RFC 9497 ristretto255/SHA-512 OPRF [12]. The suites are not interchangeable at recovery time and have different threshold-compromise semantics. Threshold Yi state directly exposes its high-entropy recovered secret. Threshold aPPSS state and public state enable unrate-limited dictionary testing, with the correct candidate yielding the recovery secret. Below reconstruction threshold, both are intended to require participation by an uncompromised online party to evaluate a candidate.

Contributions. This provisional manuscript organizes the project around four contributions:

(1) a versioned CuePolicy boundary that converts structured input into deterministic client-local bytes without persisting a cue verifier;
(2) a suite-neutral recovery composition that binds exactly one Yi or aPPSS suite to each epoch, keeps reconstruction threshold separate from authorization quorum, and feeds the recovered high-entropy value into a common HKDF/AES-256-GCM backup path;
(3) an integrated managed reference system spanning clean Clients, authenticated discovery and admission, descriptor/current-state checks, storage gateway/provider, resolver, and five party processes; and
(4) a claim-scoped evaluation methodology that binds state, flow, performance, failure, and artifact observations to the exact managed deployment without pooling historical or component evidence.

Scope. LOCUS claims no new TPASS, PPSS, aPPSS, OPRF, secret-sharing, or authenticated-encryption construction. The system tests whether explicit interfaces and service boundaries preserve the security meaning inherited from those constructions. It does not establish human memorability, production-hardening, side-channel resistance, independently administered parties, or a lifetime/global online-attempt limit.

2 Goals, System Model, and Threat Model

2.1 Security Goal

Let M be structured recovery input and let an approved policy version produce $Z_M = \mathsf{CuePolicy}_v(M)$ or fail. Let p_M be suite-bound password material, S_R the high-entropy recovery output, and B the encrypted backup. The central system goal is:

> Cloud-only, fewer-than-reconstruction-threshold party, and matching cloud-plus-below-threshold persistent states do not expose a local offline predicate for testing a candidate M'. Candidate evaluation requires online participation from sufficient authenticated threshold parties, under the selected suite's assumptions.

This is a statement about specified persistent views, not about the entropy of M, compromise of an active client, timing leakage, or the consequences of reaching the reconstruction threshold.

2.2 Entities and Trust Boundaries

Figure 1 summarizes the deployed same-host graph. Only Manager and Client pages are published on host loopback. The Manager and controller share the internal *management* network. The controller and managed Clients share the disjoint *client-lifecycle* network. Separate *manager-edge* and *browser-edge* networks publish the Manager and Client pages and do not create a container-level Manager-to-Client route. Managed Clients reach the protocol service plane, but neither browser interface contacts the provider, resolver, admission service, parties, or Docker engine directly.

2.3 Adversary Classes

Cloud/public-state adversary. The adversary obtains the encrypted backup, signed descriptor, bundle, manifest, and current-pointer state. It may delete, replay, or substitute objects. Integrity and cross-binding checks can detect malformed or inconsistent state, but availability is not guaranteed.

Below-threshold holder adversary. The adversary obtains static, read-only persistent state from fewer than k matching suite holders, possibly together with the cloud view. The claim excludes live compromise that can make an honest party evaluate arbitrary queries, side channels, volatile memory, and secret-bearing crash artifacts.

Threshold holder adversary. The adversary obtains at least k matching holder states. This is outside the main below-threshold claim. Its consequence is suite-specific: Yi exposes the shared high-entropy recovery value directly, whereas aPPSS exposes an offline dictionary predicate and yields the recovery value for a correct candidate.

Online adversary. The adversary submits candidates through admitted recovery sessions. Signed local party records provide diagnostic evidence, and authorization requires four of five authorizers, but no monotonic global authority survives coordinated rollback. Accordingly, LOCUS makes no global or lifetime attempt bound.

Control-plane adversary. The controller and Docker socket are root-equivalent trusted infrastructure. Compromise can subvert containers and therefore defeats process isolation. The Manager is less privileged but remains an availability and lifecycle control point.

Provider, resolver, and metadata observers. These roles may correlate a pseudonymous subject, backup identifier, epoch, policy and suite identifiers, memberships, object sizes, access times, and endpoints. A resolver may observe a selected query. The system aims to exclude raw cues and secret-derived verifiers from persistent state, not to provide traffic-analysis anonymity.

2.4 Assumptions

The installed application supplies authentic operator and party roots. TLS and signature algorithms operate as specified. HKDF-SHA-256 and AES-256-GCM provide their standard properties [21, 22]. At least one required online holder remains uncompromised for

Table 1: Roles in the managed reference system.

Role	Responsibility	Security boundary
User/browser	Supplies synthetic structured input and invokes enrollment or recovery through loopback pages	Browser and active managed Client jointly form the transient secret boundary; browser history, OS capture, and already loaded pages are not erased by container reset
Manager	Starts the common system and requests creation, stop, restart, kill/start, or destruction of managed Clients	Never receives cues, package bytes, suite state, recovered secrets, wrapping keys, or plaintext protected keys; has no Docker socket
Controller	Executes a fixed allowlisted set of lifecycle operations on exact project-labeled Client containers	Holds the root-equivalent Docker socket and is trusted operator infrastructure; accepts no caller-supplied image, command, mount, host path, network, or arbitrary identifier
Managed Client	Runs the thin UI and client state machines, performs policy processing, suite operations, backup encryption/decryption, and package validation	Only active Client may transiently handle the complete secret path; no protocol state is shared between Client instances
Operator/discovery	Signs descriptors and current pointers and returns public configuration	Operator signing is not a self-authenticating trust root; Clients begin with installed trust and verify live party summaries
Admission	Issues short-lived, proof-key-bound capabilities for exact operations	Independent of cues and suite correctness; admission availability is a recovery prerequisite, not a hidden recovery factor
Storage gateway/provider	Maps admitted operations to immutable backups, descriptors, bundles, and a mutable current pointer	Provider credentials remain gateway-side; Clients cannot list a bucket or receive storage credentials
Resolver	Implements the policy-specific resolution contract when required	May learn lookup input; local hashing after lookup does not hide it
Five parties	Every party authorizes; the selected subset also holds Yi or aPPSS state	Reconstruction is 2-of-3 or 3-of-5, while authorization is separately typed as 4-of-5

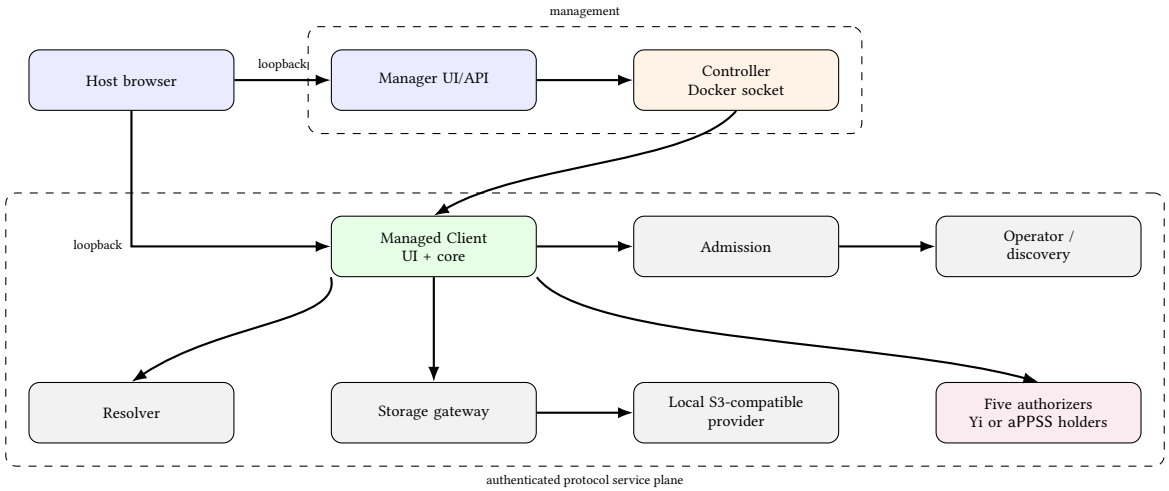

Figure 1: Managed integrated reference system. Network boxes are conceptual: all roles execute on one Docker engine under one operator. This demonstrates process, credential, and network separation, not host separation or independent administration.

the below-threshold claim, and the active Client is trusted while it processes secrets. Suite-specific claims inherit the source construction, random-oracle, OPRF, group, and corruption assumptions; implementation tests are not cryptographic proofs.

3 Design Requirements and Invariants

Deterministic policy boundary. Every policy declares accepted structure, cardinality, resolver behavior, canonicalization, ambiguity, duplicate, and version rules. It returns exactly one byte string or fails. It must not emit hints, fuzzy alternatives, or persisted candidate verifiers.

One suite per epoch. Enrollment or successor creation explicitly selects one registered suite and topology. The authenticated descriptor binds that choice. Recovery derives the adapter only from verified configuration: no automatic fallback, trial of another suite, cross-suite share mixing, or in-place conversion is allowed.

Threshold separation. The type for suite reconstruction k-of-n is distinct from the authorization quorum. The implemented profiles are Yi and aPPSS at 2-of-3 and 3-of-5 over five authorizers, with authorization fixed at 4-of-5.

Protected path. The active Client alone executes

$$M \xrightarrow{\text{CuePolicy}_v} Z_M \xrightarrow{\text{suite domain}} p_M \xrightarrow{\text{Setup/Recover}} S_R \xrightarrow{\text{HKDF-SHA-256}} K_{\text{wrap}} \xrightarrow{\text{AES-256-GCM}} sk.$$

Raw cues, selected canonical descriptors, Z_M, p_M, S_R, K_{wrap}, and plaintext sk must not persist in cloud, party, Manager, controller, package, or retained evidence state.

Authenticated bootstrap. A descriptor cannot authenticate its own trust root. Installed trust authenticates the discovery endpoint, operator key, party identities, and endpoints. A clean Client verifies signed current state and obtains a fresh 4-of-5 matching party-current quorum before cue-dependent work.

Atomic lifecycle. A predecessor remains recoverable until successor party state, backup, descriptor, bundle, and current pointer are durable and the successor has recovered the same protected key. Retry behavior is idempotent and bound to exact request/configuration digests.

Evidence isolation. Historical component measurements, frozen deployments, unit vectors, and P6 process profiles remain controls. Principal system evidence must exercise the exact Manager-created Client graph and bind source state, resolved configuration, images, identities, networks, provider, suite, topology, policy, failure schedule, and Client instances.

4 Architecture

4.1 CuePolicy and Resolver Registry

The CuePolicy interface receives structured input plus registered public parameters and returns canonical bytes Z_M or a typed failure. The registry preserves the original exactly-three location–person-pair policy byte for byte. Additional policies demonstrate interface generality: exactly three distinct quantized coordinates, canonical E.164 phone numbers, or constrained canonical email addresses. Direct-input policies use a *NoResolver* adapter; the composite location–person policy uses deterministic resolution in the reproducible profile.

Canonicalization is intentionally strict. Ordering is deterministic; duplicate semantic values, ambiguous records, unsupported versions, out-of-range coordinates, and resolver drift fail rather than generating candidate lists. Public policy metadata identifies the rules, but no cue hash or selected descriptor is retained as a recovery hint.

4.2 RecoveryDescriptor, Bundle, and Discovery

The signed *RecoveryDescriptor* binds:

- pseudonymous subject, backup identifier, epoch, and recovery identity;
- exact encrypted-backup member digest and byte length;
- policy identifier, opaque public parameters, and resolver profile;
- one suite identifier, public state, typed k, n, and holder mapping;
- five authorizer identities/endpoints, 4-of-5 authorization, admission profile, audience, and operation namespace; and
- configuration digest and predecessor binding.

It excludes raw cues, selected record identifiers, cue digests, candidate hints, p_M, suite secret state, S_R, K_{wrap}, plaintext private keys, credentials, and a trust root that originates only in the descriptor.

Each immutable recovery bundle is a bounded, deterministic ZIP containing exactly `backup.json`, `descriptor.json`, and `manifest.json`. A signed current pointer outside the ZIP binds the active bundle locator, bundle digest and length, descriptor digest, epoch, and configuration digest. Exact-member, size, compression, path, canonical encoding, signature, digest, and cross-object validation occurs before cue-dependent work.

The default discovery profile uses admission to obtain a short-lived proof-key-bound capability scoped to one pseudonymous subject, backup, operation, audience, nonce, and expiry. The storage gateway performs an exact object operation without returning provider credentials or general listing authority. An optional signed recovery receipt may carry a discovery handle and initial public anchor, but it contains neither cues nor credentials and is not a freshness witness.

4.3 Managed Clients and Recovery Packages

Normal startup creates the service plane, Manager, and controller but no Client. The Manager requests a Client through the controller's fixed API. The controller assigns a public instance identifier and publishes a separate loopback UI port. Enrollment executes entirely inside that Client/browser boundary.

An authenticated client recovery package transfers the bounded public configuration needed by another clean Client. The package is untrusted input: its digest/signature, current pointer, descriptor, bundle, party identities, and current-party summaries are all revalidated. The package cannot override suite, policy, membership, endpoint, threshold, or fallback behavior.

Stop/start, restart, and kill/start are destructive volatile resets. They retain the public Client instance identifier but rotate the process proof key and clear server-side key slots, package caches, operations, and sessions. Destroy removes both container and instance identifier. A later creation receives a new identifier. These semantics do not erase browser memory or an already loaded document.

4.4 Bootstrap and Credential Lifetime

A networkless one-shot bootstrap creates synthetic service credentials, public configuration, empty per-role roots, and fixtures. It must not inject suite state, cues, protected keys, or secret-bearing Client state. The bootstrap container runs as root with all capabilities dropped except `CHOWN` and `DAC_READ_SEARCH`, has no Docker socket, and exits before unprivileged runtime services start. Its CA is valid for 366 days and leaf certificates for 365 days; there is no in-place renewal.

Normal Manager stop and emergency integrated cleanup preserve exact-project volumes. Only an explicit reset-state operation removes all role/provider volumes and credentials. That operation irreversibly destroys the local trust domain and the deployment state required by earlier packages.

5 Protocol Construction

5.1 Common Inputs and Domain Separation

For backup identifier b, epoch e, policy v, selected suite σ, topology (k, n), and authenticated membership $\mathcal{P}$, the Client validates an immutable public context ctx. It computes

$$Z_M = \mathsf{CuePolicy}_v(M)$$

or terminates before contacting suite holders. Suite-specific framing derives

$$p_M = \mathsf{H}(\mathsf{domain}_\sigma \parallel \mathsf{Encode}(ctx) \parallel \mathsf{Encode}(Z_M)).$$

The Yi and aPPSS domains and encodings are disjoint. Public configuration commits to the exact context so state from a different backup, epoch, policy, suite, topology, or membership cannot be combined.

After successful setup or recovery, the suite yields S_R. The common backup path derives

$$K_{\text{wrap}} = \mathsf{HKDF}_{\text{SHA256}}(S_R,\ salt,\ info(ctx))$$

and encrypts the private key using AES-256-GCM:

$$(c, tag) = \mathsf{AEAD.Enc}(K_{\text{wrap}}, nonce, sk, \mathrm{AAD}(ctx)).$$

The backup object carries the ciphertext, nonce, tag, and public bindings. It does not carry a cue verifier.

5.2 Frozen Yi TPASS Suite

The Yi adapter preserves the frozen Ristretto255 wire formats and equations. During setup, the active Client samples a high-entropy group secret and distributes native TPASS state across the selected holders. During recovery, the candidate p'_M is mapped through the frozen suite domain, holders produce authenticated protocol responses, and a valid k-subset lets the Client reconstruct the group secret when the candidate is correct.

The important compromise distinction is that matching threshold Yi state contains shares sufficient to interpolate the password scalar, secret exponent, and digest. Thus a threshold persistent-state compromise directly exposes the high-entropy recovery output; it is not described as merely an offline dictionary attack. Fewer than k matching states are covered by the inherited TPASS threshold claim, subject to the concrete mapping and review assumptions.

5.3 Augmented PPSS Suite

The aPPSS profile instantiates only Section 3, Figure 4, and Theorem 2 of *Password-Protected Threshold Signatures*; it excludes the paper's threshold-signature, aptSIG, BLS, and PFS layers. It uses security parameter $\lambda = 128$, Shamir sharing over polynomial-basis $\mathrm{GF}(2^{128})$, and one independently keyed RFC 9497 ristretto255/SHA-512 OPRF instance per holder.

For holder i, the Client obtains an OPRF value on the framed p_M and derives a 16-byte mask ρ_i. Setup samples $s \leftarrow \mathrm{GF}(2^{128})$, shares s with a degree-$(k-1)$ polynomial to obtain s_i, and stores

$$e_i = s_i \oplus \rho_i$$

as public masked-share material. A domain-separated SHA-256 call over the context, p_M, complete canonical e, and encoded s is split into a 16-byte commitment C and the 16-byte recovery secret S_R. The Client uses this S_R directly in the common HKDF path; it does not create or share an additional unmasked recovery secret.

For recovery, each selected holder evaluates its bound OPRF instance. The Client unmasks k shares, interpolates s', recomputes the hash, and accepts only if the commitment matches. Wrong input, invalid OPRF output, reconstruction failure, commitment mismatch, and final backup failure collapse to a generic secret-dependent rejection when availability can still be distinguished safely. Because the base profile uses non-verifiable OPRF mode, a malicious selected server can cause an unattributable abort.

Matching threshold aPPSS holder keys plus public (e, C) permit local, unrate-limited testing of candidates: incorrect candidates fail C, while the correct candidate yields S_R. Below threshold, candidate testing still requires an uncompromised holder's online OPRF participation under the construction assumptions.

5.4 Enrollment

Enrollment proceeds as follows:

(1) The Client obtains admission and validates installed trust, the exact party directory, policy, suite, topology, and authorization profile.
(2) The CuePolicy canonicalizes M, and the selected suite creates fresh per-epoch state through authenticated provisioning routes.
(3) The suite yields S_R; the Client encrypts the protected key and verifies its stable key identity.
(4) The operator produces the signed descriptor; the Client constructs and locally revalidates the deterministic bundle.
(5) The storage gateway creates the immutable bundle and installs the authenticated current pointer with compare-and-swap semantics.
(6) Required parties attest readiness. The Client exports a bounded recovery package only after durable completion.

Every mutating route is request-bound and idempotent. A retry with identical bytes returns the same public result; a colliding request identifier with different bytes fails.

5.5 Clean-Client Recovery

A clean Client begins without enrollment-client state. It receives installed trust, a fresh proof key, a recovery handle or package, and transient user input. It then:

(1) strictly decodes the package and obtains admitted discovery;
(2) authenticates the current pointer, bundle, descriptor, manifest, and backup cross-bindings using installed operator trust;
(3) matches descriptor endpoints and keys against installed party trust;
(4) obtains four matching, short-lived party-current summaries and rejects mixed or stale epochs before processing cues;
(5) invokes the descriptor-bound CuePolicy and the one descriptor-bound recovery suite, with no selector or fallback;
(6) reconstructs S_R, derives K_{wrap}, authenticates and decrypts the backup, and verifies protected-key identity; and
(7) returns the private key only inside the active Client/browser boundary.

Public parsing, authentication, freshness, and availability failures remain typed. Secret-dependent failures are deliberately coarsened to reduce candidate feedback.

5.6 Successor Epochs

A suite or policy change creates a fresh successor epoch. The durable coordinator prepares new party state, encrypts a successor backup, signs and publishes a new descriptor and bundle, recovers the protected key through the prepared successor, atomically changes the current pointer, activates the successor, and only then retires the predecessor. Same-suite and cross-suite transitions are supported, but state is never translated or mixed; a cross-suite transition is a complete new setup around the recovered protected key.

6 Security Analysis

6.1 Persistent-State Views

Table 2 states the intended result for the principal static views. The table is not a new proof: rows inherit their suite assumptions and are supported operationally only at the exact evidence scope.

6.2 Why Public Metadata Is Not a Cue Verifier

The descriptor and pointer expose policy/suite names, topology, membership, endpoints, public state, object lengths, digests, epochs, and pseudonymous identifiers. Their ordinary SHA-256 digests bind already public canonical objects. No digest is computed directly over M, selected resolver records, Z_M, p_M, S_R, or K_{wrap}. Consequently, copying the cloud view does not by itself supply a function that distinguishes a candidate without the suite interaction. This argument assumes correct domain separation, canonical decoding, and absence of secret-bearing logs or crash artifacts.

6.3 Descriptor Authenticity and Rollback

A valid signature proves issuance, not freshness. The clean Client begins from installed trust, authenticates the operator-signed current pointer and descriptor, pins the party directory, and requires four matching short-lived party-current summaries. This detects a stale cloud view when sufficient parties report a different current epoch. It does not defeat coordinated restoration of cloud state, the required party states and keys, Client clock, and installed-trust view. A monotonic external witness or consensus service would define a separate architecture.

6.4 Authorization Is Not Reconstruction

Four authorizers must approve the operation even when only two or three holders reconstruct the suite value. The authorization quorum limits which requests reach secret-dependent processing and keeps operation policy separate from cryptographic threshold semantics. It is not a substitute for a global attempt counter: signed local audit records can be rolled back together.

6.5 Active-Client and Controller Boundaries

An active Client compromise reveals input and recovered key material and is outside the principal snapshot claim. The Manager never handles these values, but the controller's Docker authority can subvert Client isolation and is trusted. Network segmentation narrows accidental routes and makes the intended graph testable; it does not protect against a malicious host administrator or kernel.

6.6 Failure Channels

Before cue processing, bounded errors distinguish malformed, unauthenticated, expired, stale, unavailable, or configuration-mismatch conditions so operators can repair availability failures. After candidate evaluation begins, the client-facing result coarsens wrong input, suite reconstruction failure, commitment failure, and backup-authentication failure. The evaluation verifies categories and retained output safety, but does not establish constant-time execution or eliminate all network timing channels.

7 Implementation

The active implementation is self-contained under `prototype_final/`. It does not import runtime source, scripts, tests, deployment assets, or state from the historical root prototype. A Python orchestration and service layer implements state machines, strict codecs, admission, discovery, storage, lifecycle, Manager/controller APIs, and evidence collectors. Native Rust components implement the Ristretto255 Yi core and suite primitives. Standard HKDF-SHA-256 and AES-256-GCM protect backups.

The reproducible deployment uses Docker Compose for the fixed service plane and a constrained Docker API for dynamic Clients. Mutual TLS and pinned service identities authenticate internal routes. The provider is local S3-compatible object storage behind the application gateway. A deterministic resolver and synthetic admission issuer avoid external accounts. The only published interfaces are the Manager and separately created Client loopback pages.

The Client UI is a thin semantic HTML/CSS/JavaScript caller. Policy canonicalization, suite selection, descriptor validation, cryptography, and lifecycle ordering remain in the Client API. Telemetry and remote assets are disabled; secret-bearing browser persistence, logs, screenshots, and crash output are prohibited. A synthetic private key may be displayed transiently only when explicitly requested inside the active Client/browser boundary.

The operator surface is intentionally small:

- `integrated-check` and `integrated-config`;
- `integrated-start` and `integrated-stop`; and
- `integrated-smoke`.

Later evidence commands are assigned by their specific gates. Normal operation starts without a mode and proceeds through the Manager and Client pages. Emergency stop preserves state unless the operator explicitly requests the irreversible reset-state behavior.

8 Evaluation Methodology

8.1 Evaluation Questions

The evaluation separates five questions:

Q1: Do strict decoders and state machines reject malformed, ambiguous, replayed, stale, mixed, or colliding inputs?

Q2: Do persistent role snapshots contain only the state allowed for each role, and do below-threshold views lack a locally observable candidate signal under bounded networkless tests?

Table 2: Suite-specific persistent-state consequences.

Adversary view	Yi consequence	aPPSS consequence
Cloud/public state only	Encrypted backup and authenticated public configuration provide no local candidate predicate	Public (e, C), descriptor, and encrypted backup provide no local candidate predicate without holder OPRF keys
Fewer than k matching holder states	No reconstruction and no local candidate predicate under the inherited Yi threshold assumptions	No local candidate predicate; testing requires an uncompromised holder's online OPRF participation under the aPPSS assumptions
Cloud plus fewer than k matching holder states	Storage separation adds no missing threshold state, so the combined view remains below threshold	Ciphertext and public commitment add no missing OPRF key; the combined view remains below threshold
At least k matching holder states	Direct interpolation exposes the high-entropy recovery output	Public state plus threshold holder keys enables unrate-limited offline candidate testing; a correct candidate yields the recovery output

Q3: Does the managed Client contact only the expected authenticated service graph without retaining payloads in flow evidence?

Q4: What end-to-end latency and role-storage cost does the exact same-host managed system exhibit across suites, topologies, and one-holder failure?

Q5: Can an independent reviewer reproduce the artifact and confirm the claim-critical paper-to-profile-to-code mappings?

8.2 Assurance Controls

Unit, property, malformed-input, concurrency, stale-state, exact-retry, crash, and complete managed smoke tests are implementation controls. The smoke gate exercises four suite/topology arms, all 26 valid threshold subsets (three 2-of-3 subsets for each suite and ten 3-of-5 subsets for each suite), four clean Clients, restart preservation, destructive reset, state audits, output scans, and exact cleanup. These tests establish behavior of the tested build, not cryptographic proof or retained system evidence.

8.3 Persistent-State Evidence

The retained managed-state corpus fixes scenarios SB01–SB14, four snapshot points, exact role views, positive controls, aggregate-only observations, and prohibited-output scans. It contains 42 reports: 18 Yi, 18 aPPSS, and six common managed-control reports. Suite/topology paths remain disjoint. Reports bind the managed manifest, source commit, public configuration, role set, snapshot point, and privacy-safe aggregate file/byte counts. They exclude files, databases, credentials, keys, shares, cues, candidates, logs, absolute paths, secret-state digests, and per-candidate outcomes.

The corpus is complete and hash-closed. Within its static, same-host, networkless boundaries, all scheduled reports and positive controls validated. This supports implementation-level statements about the enumerated persistent surfaces. It does not prove source-suite theorems, adaptive security, side-channel resistance, or absence of unexamined volatile state.

8.4 Payload-Free Flow Evidence

The managed-flow corpus fixes NF01–NF12 over all four arms and common Manager/controller operations. It contains 30 reports: 12 Yi, 12 aPPSS, and six common. Temporary instrumentation records only bounded application role and category counters and body-byte totals. It retains no route, address, header, payload, packet capture, timing, ordering, cue, key, credential, or session trace. The temporary stream is scanned, reconciled against the resolved graph, and discarded before publication.

All scheduled contacts and forbidden-edge checks validated for the observed application boundary. This demonstrates the intended managed service graph at that instrumentation point. It is not packet-level evidence and does not establish host separation, traffic confidentiality against the host, or independent administration.

8.5 Attempt-Control Boundary

A bounded model and integrated wrapper reproduce seven attempt-accounting scenarios on the five-authorizer/4-of-5 configuration. Three quorum-only scenarios retain rollback counterexamples. Signed local certificates make local histories inspectable but cannot create a monotonic fact after all relevant state is restored. The result is intentionally negative: LOCUS does not claim a rollback-resistant global or lifetime attempt bound.

8.6 Managed Performance Study

The affordable performance profile uses the exact D025 graph, one common image/source state, local S3-compatible provider, five authorizers, and 4-of-5 authorization. Four matched arms cross suite and topology:

(1) Yi 2-of-3 with canonical-email/NoResolver;
(2) aPPSS 2-of-3 with canonical-email/NoResolver;
(3) Yi 3-of-5 with location–person/deterministic resolution; and
(4) aPPSS 3-of-5 with location–person/deterministic resolution.

Three deterministic blocks use 12 fresh projects. AP00–AP06 expand to 324 scheduled slots: 12 warm-ups and 312 measured observations. The five central scenarios measure enrollment, successful recovery, wrong-input rejection, recovery with one selected holder unavailable, and clean-Client package recovery; a structural snapshot records aggregate application and persisted role bytes. Client monotonic time is the end-to-end observation clock, with applicable phase durations. Results remain separated by suite, topology, scenario, block, and validity.

For each of 24 groups, processing reports count, minimum, first quartile, median, third quartile, maximum, and a secondary arithmetic mean. Quartiles use the specified type-7 rule. No outlier is removed, no confidence interval or hypothesis test is performed, and 12 matched median pairs are presented side-by-side without pooling or suite-advantage inference.

Interpretation boundary. The study is descriptive same-host engineering evidence. It cannot support scalability, production capacity,

Table 3: Provisional managed performance table. Values will be inserted only from the validated, hash-closed v2 retained corpus.

Arm	Scenario	*n*	Median (ms)	Q1–Q3 (ms)	Min–max (ms)	Invalid
Yi 2-of-3	Enrollment	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
Yi 2-of-3	Successful recovery	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
aPPSS 2-of-3	Enrollment	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
aPPSS 2-of-3	Successful recovery	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
Yi 3-of-5	Enrollment	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
Yi 3-of-5	Successful recovery	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
aPPSS 3-of-5	Enrollment	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
aPPSS 3-of-5	Successful recovery	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**

Table 4: Provisional closure fields.

Closure item	Status in this draft
Independent Yi mapping confirmation	**TBD**
Independent aPPSS mapping confirmation	**TBD**
Common-composition confirmation	**TBD**
Clean-host artifact reproduction	**TBD**
Portable artifact identifier and digest	**TBD**
Managed performance v2 corpus digest	**TBD**

wide-area latency, multi-host availability, independent administration, human-task time, usability, or a claim that one suite is faster in general. The old 30-sample component measurements are not pooled with this design.

8.7 Independent Review and Artifact Reproduction

The final evaluation requires two distinct independent checks. A qualified cryptographic reviewer must confirm the claim-critical Yi and aPPSS source-to-profile-to-code mappings and the common LOCUS composition. A systems reviewer must reproduce the managed deployment and evidence gates on a clean host from the portable artifact. The final manuscript should report reviewer qualifications, reviewed commits, deviations, resolutions, artifact digest, environment, and pass/fail status.

9 Limitations

No human study. The system does not establish cue entropy, memorability, delayed reproducibility, accessibility, error rates, enrollment burden, or recovery success for real users. The reference inputs are synthetic. Public or socially known information may make a user's choices highly predictable.

Inherited cryptography and pending review. Yi TPASS and aPPSS are inherited constructions. The concrete mappings add engineering choices, domains, encodings, and failure behavior. Tests cannot replace independent review or source proofs. Until the closure fields are filled, statements that rely on the inherited mapping remain provisional.

Threshold compromise. The primary claim ends below reconstruction threshold. Threshold Yi state directly exposes the recovery output. Threshold aPPSS state enables offline dictionary testing. Neither suite protects against compromise of an active Client that handles the complete secret path.

Same host and one operator. Containers, networks, credentials, and processes are separated on one Docker engine under one administration domain. This is not multi-host separation, fault independence, legal independence, or independent administration. The controller and host are trusted.

Attempt control and rollback. The design has signed local accounting but no globally monotonic authority. Coordinated rollback can restore an earlier attempt state. Party-current summaries detect many stale-cloud cases but not a completely consistent rollback of all authoritative state and trust.

Availability. Admission, discovery, storage gateway/provider, the authorization quorum, and the selected recovery threshold are prerequisites. Deletion, outage, credential expiry, insufficient parties, or loss of installed trust can block recovery. The managed credentials have bounded lifetime and no in-place renewal.

Metadata and resolver privacy. Operators and parties observe pseudonymous identifiers, configuration, membership, endpoints, sizes, timing, and access patterns. A resolver may see the submitted lookup. The flow corpus is payload-free but is not an anonymity or traffic-analysis study.

Operational completeness. General party replacement, arbitrary k, n, secure hardware, production key ceremonies, disaster recovery across regions, credential renewal, complete secure erasure, unbounded concurrency, and adversarial side-channel testing are outside the implemented profile.

Provider and deployment scope. The reproducible result uses local S3-compatible storage. A real AWS adapter is supplemental and separately gated. Local functional behavior cannot be generalized to a real provider, WAN, or production account.

10 Related Work

Secret sharing and distributed recovery. Shamir secret sharing distributes a high-entropy secret but does not by itself make low-entropy recovery input safe [24]. Verifiable and asynchronous variants strengthen correctness and availability under different models [7, 15, 23]. Social-wallet and decentralized recovery systems distribute authorization among guardians or contracts, with different trust and privacy tradeoffs [10, 17, 28]. LOCUS instead focuses on the composition of structured human input, password-authenticated threshold recovery, encrypted object storage, and a clean replacement-client workflow.

Password-protected secret sharing. PPSS and TPASS constructions seek to prevent offline password testing below a defined corruption threshold [3, 18, 19, 26, 27]. Memento and later password-protected retrieval systems explore related hostile storage and hardware-assisted settings [8, 14]. LOCUS does not improve their cryptographic proofs. It exposes two inherited suites behind explicit, non-fallback interfaces and evaluates whether the surrounding descriptor, discovery, storage, UI, and lifecycle system preserves their relevant persistent-state boundary.

Encrypted backup and service-scale recovery. SafetyPin distributes trust for PIN-protected backups, while SVR3 reports a service-scale key-recovery design [9, 11]. Analyses of encrypted cloud-backup systems show that protocol details around credentials, metadata, and server compromise matter [13]. LOCUS contributes a reproducible same-host reference graph and a narrow storage-separated cue-testing claim, not a service-scale deployment.

Human-derived recovery input. Personal knowledge, graphical passwords, and location-based schemes expose predictability, observation, and reproduction risks [1, 4, 5, 16, 25]. Fuzzy commitments and fuzzy extractors address noisy sources under their own entropy and helper-data assumptions [6, 20]. LOCUS instead makes canonicalization strict and rejects ambiguity; it does not claim tolerance or human usability.

11 Conclusion

LOCUS shows how deterministic structured-input processing can be composed with selectable inherited threshold suites and storage-separated encrypted backup in a complete replacement-client system. The central design discipline is explicit separation: cue processing stays client-local, each epoch binds one suite without fallback, reconstruction and authorization thresholds remain different types, the provider stores no suite state, parties store no encrypted private key, and the Manager/controller remain outside the protected path.

The managed reference implementation makes these boundaries executable across admission, discovery, storage, resolution, five parties, dynamic Clients, and lifecycle operations. Retained state and flow corpora provide scoped evidence for the exact same-host graph. Performance, independent cryptographic review, and clean-host artifact closure remain explicit provisional fields in this personal draft. Even after those fields close, the appropriate conclusion is limited: under the selected suite assumptions, the evaluated cloud-only and below-threshold persistent views do not create a local offline cue-testing predicate. The result does not imply memorable cues, production security, global rate limiting, or independence among parties.

References

[1] Mahdi Nasrullah Al-Ameen, Kanis Fatema, Matthew Wright, and Shannon Scielzo. 2015. The Impact of Cues and User Interaction on the Memorability of System-Assigned Recognition-Based Graphical Passwords. In *Eleventh Symposium on Usable Privacy and Security (SOUPS 2015)*. USENIX Association, Ottawa, Canada, 185–196. https://www.usenix.org/conference/soups2015/proceedings/presentation/al-ameen

[2] Mahdi Nasrullah Al-Ameen and Matthew Wright. 2017. Exploring the Potential of GeoPass: A Geographic Location-Password Scheme. *Interacting with Computers* 29, 4 (2017), 605–627. doi:10.1093/iwc/iww033

[3] Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. 2011. Password-Protected Secret Sharing. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*. ACM, 433–444. doi:10.1145/2046707.2046758

[4] Robert Biddle, Sonia Chiasson, and Paul C. van Oorschot. 2012. Graphical Passwords: Learning from the First Twelve Years. *Comput. Surveys* 44, 4 (2012), 1–41. doi:10.1145/2333112.2333114

[5] Joseph Bonneau, Elie Bursztein, Ilan Caron, Rob Jackson, and Mike Williamson. 2015. Secrets, Lies, and Account Recovery: Lessons from the Use of Personal Knowledge Questions at Google. In *Proceedings of the 24th International Conference on World Wide Web (WWW)*. ACM, 141–150. doi:10.1145/2736277.2741691

[6] Xavier Boyen. 2004. Reusable Cryptographic Fuzzy Extractors. In *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*. 82–91. doi:10.1145/1030083.1030096

[7] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. 2002. Asynchronous Verifiable Secret Sharing and Proactive Cryptosystems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*. ACM, 88–97. doi:10.1145/586110.586124

[8] Jan Camenisch, Anja Lehmann, Anna Lysyanskaya, and Gregory Neven. 2014. Memento: How to Reconstruct Your Secrets from a Single Password in a Hostile Environment. In *Advances in Cryptology – CRYPTO 2014 (Lecture Notes in Computer Science, Vol. 8617)*. Springer, 256–275. doi:10.1007/978-3-662-44381-1_15

[9] Graeme Connell, Vivian Fang, Rolfe Schmidt, Emma Dauterman, and Raluca Ada Popa. 2024. Secret Key Recovery in a Global-Scale End-to-End Encryption System. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 703–719. https://www.usenix.org/conference/osdi24/presentation/connell

[10] Weiqi Dai, Yan Lv, Kim-Kwang Raymond Choo, Zhongze Liu, Deqing Zou, and Hai Jin. 2021. CRSA: A Cryptocurrency Recovery Scheme Based on Hidden Assistance Relationships. *IEEE Transactions on Information Forensics and Security* 16 (2021), 4291–4305. doi:10.1109/TIFS.2021.3104142

[11] Emma Dauterman, Henry Corrigan-Gibbs, and David Mazières. 2020. SafetyPin: Encrypted Backups with Human-Memorable Secrets. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 1121–1138. https://www.usenix.org/conference/osdi20/presentation/dauterman-safetypin

[12] Alex Davidson, Armando Faz-Hernandez, Nick Sullivan, and Christopher A. Wood. 2023. Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups. RFC 9497. doi:10.17487/RFC9497

[13] Gareth T. Davies, Sebastian H. Faller, Kai Gellert, Tobias Handirk, Julia Hesse, Máté Horváth, and Tibor Jager. 2023. Security Analysis of the WhatsApp End-to-End Encrypted Backup Protocol. In *Advances in Cryptology — CRYPTO 2023*. Springer, 330–361. doi:10.1007/978-3-031-38551-3_11

[14] Sebastian Faller, Tobias Handirk, Julia Hesse, Máté Horváth, and Anja Lehmann. 2024. Password-Protected Key Retrieval with(out) HSM Protection. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2445–2459. doi:10.1145/3658644.3690358

[15] Paul Feldman. 1987. A Practical Scheme for Non-Interactive Verifiable Secret Sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS)*. 427–438. doi:10.1109/SFCS.1987.4

[16] Alina Hang, Alexander De Luca, Matthew Smith, Michael Richter, and Heinrich Hussmann. 2015. Where Have You Been? Using Location-Based Security Questions for Fallback Authentication. In *Eleventh Symposium on Usable Privacy and Security (SOUPS 2015)*. USENIX Association, Ottawa, Canada, 169–183. https://www.usenix.org/conference/soups2015/proceedings/presentation/hang

[17] Allan B. Pedin IV, Nazli Siasi, and Mohammad Sameni. 2023. Smart Contract-Based Social Recovery Wallet Management Scheme for Digital Assets. In *Proceedings of the 2023 ACM Southeast Conference (ACMSE)*. ACM, 177–181. doi:10.1145/3564746.3587016

[18] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. 2014. Round-Optimal Password-Protected Secret Sharing and T-PAKE in the Password-Only Model. In *Advances in Cryptology – ASIACRYPT 2014 (Lecture Notes in Computer Science, Vol. 8874)*. Springer, 233–253. doi:10.1007/978-3-662-45608-8_13

[19] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. 2016. Highly-Efficient and Composable Password-Protected Secret Sharing (Or: How to Protect Your Bitcoin Wallet Online). In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 276–291. doi:10.1109/EuroSP.2016.30

[20] Ari Juels and Martin Wattenberg. 1999. A Fuzzy Commitment Scheme. In *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS)*. 28–36. doi:10.1145/319709.319714

[21] Hugo Krawczyk and Pasi Eronen. 2010. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869. doi:10.17487/RFC5869

[22] David McGrew. 2008. An Interface and Algorithms for Authenticated Encryption. RFC 5116. doi:10.17487/RFC5116

[23] Berry Schoenmakers. 1999. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic Voting. In *Advances in Cryptology — CRYPTO'99 (Lecture Notes in Computer Science, Vol. 1666)*. Springer, 148–164. doi:10.1007/3-540-48405-1_10

[24] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. doi:10.1145/359168.359176
[25] Jeff Yan, Alan Blackwell, Ross Anderson, and Alasdair Grant. 2004. Password Memorability and Security: Empirical Results. *IEEE Security & Privacy* 2, 5 (2004), 25–31. doi:10.1109/MSP.2004.81
[26] Xun Yi, Feng Hao, Liqun Chen, and Joseph K. Liu. 2015. Practical Threshold Password-Authenticated Secret Sharing Protocol. In *Computer Security – ESORICS 2015 (Lecture Notes in Computer Science, Vol. 9326)*. Springer, 347–365. doi:10.1007/978-3-319-24174-6_18
[27] Xun Yi, Zahir Tari, Feng Hao, Liqun Chen, Joseph K. Liu, Xuechao Yang, Kwok-Yan Lam, Ibrahim Khalil, and Albert Y. Zomaya. 2019. Efficient Threshold Password-Authenticated Secret Sharing Protocols for Cloud Computing. *J. Parallel and Distrib. Comput.* 128 (2019), 57–70. doi:10.1016/j.jpdc.2019.01.013
[28] Yanlin Zhu, Lirong Xia, and Oshani Seneviratne. 2019. A Proposal for Account Recovery in Decentralized Applications. In *2019 IEEE International Conference on Blockchain (Blockchain)*. 148–155. doi:10.1109/Blockchain.2019.00028

A Provisional Artifact and Reproducibility Contract

The final artifact should contain the self-contained `prototype_final/` source, frozen dependency lock, strict schemas, public vectors, managed deployment manifests, retained aggregate corpora, processors, and an allowlisted command surface. It must exclude credentials, certificates, private keys, databases, generated role state, service logs, screenshots, traces, developer paths, and build directories.

A clean reviewer workflow should:

(1) synchronize frozen dependencies and run the integrated static/check gate;
(2) validate the resolved managed graph and its prohibited topology;
(3) start the Manager, create Clients, and exercise all four arms;
(4) run the complete disposable smoke and output-safety checks;
(5) validate the retained P8 and P9 corpus manifests without regenerating secret-bearing snapshots; and
(6) stop exact-project resources and confirm cleanup.

The final package identifier, source commit, archive digest, host requirements, expected duration, and independently reproduced status are **TBD**.

B Suite Mapping Notes

B.1 Yi Mapping Boundary

The Yi implementation preserves the frozen Ristretto255 profile, canonical wire formats, group validation, hash domains, and recovery equations. The suite adapter changes orchestration but not the frozen mathematical construction. Claim-critical review must confirm threshold notation, share reconstruction, transcript binding, zero-knowledge checks, and the statement that threshold persistent state directly reconstructs the high-entropy recovery output.

B.2 aPPSS Mapping Boundary

The aPPSS mapping fixes $t_{\text{paper}} = k - 1$, so paper reconstruction threshold $t + 1$ becomes LOCUS k. The implemented profiles are $(k, n) = (2, 3)$ and $(3, 5)$. The field is $\mathrm{GF}(2^{128})$ with modulus $x^{128} + x^7 + x^2 + x + 1$, canonical 16-byte big-endian field encodings, and integer party coordinates. Every holder has an independent, nonzero, per-epoch OPRF key.

The epoch context binds suite identifier, backup, epoch, policy, threshold, membership, and authenticated service identities. A separate post-setup configuration digest binds the resulting public state and encrypted backup; the two digests are deliberately acyclic and non-interchangeable. The Figure-4 random-oracle operation is concretized with framed SHA-256 and split into a 16-byte commitment and 16-byte recovery secret. This is an engineering instantiation assumption, not a proof of Theorem 2.

C Manuscript Completion Checklist

Before this personal draft could become a candidate authoritative manuscript, the following items would require separate project approval:

- replace every **TBD** performance cell from the validated retained corpus and report invalid observations;
- insert independent-review identities or anonymized qualifications, reviewed commits, findings, deviations, and resolutions;
- insert the clean-host artifact identifier, digest, environment, and reproduction result;
- synchronize the official claim/evidence matrix, threat model, limitations, generated tables, and artifact documentation;
- verify every citation, add the formal aPPSS primary-source bibliography entry, and remove provisional language only where evidence permits; and
- obtain a separate owner-approved delta before changing the authoritative manuscript.